

Notes on Bringing up the H8 Mini

*Glenn Roberts
September 2026*

In the summer of 2026 Lee Hart began developing a miniaturized version of the venerable Heathkit H8 computer, in honor of the upcoming 50th anniversary of its unveiling. To fit the H8 functionality into a small (roughly 3" x 6") form factor Lee had to vary the design from the original, most significantly by using an 8255 Programmable Peripheral Interface (PPI) chip for all input/output functions. This necessitated the development of new firmware.

I have been part of a team that did the initial work to bring this device to life, contributing primarily to development and testing of the firmware. These notes document my thoughts, techniques and lessons learned as I have gone through this process. My work has been on Revision A of the board, though Lee has already been working on Revision B.

Configuring the H8 Mini testbed

I assembled the H8 Mini (Rev. A) using Lee's instructions. The kit was supplied with a 27C256 32K EPROM, however it can be configured to use a 39SF040 (512K byte) EEPROM. Since I knew I would be doing many iterations of ROM programming I chose to use the 39SF040. As of this writing these are available from DigiKey (part #SST39SF040-70-4C-PHE-ND; \$3.87, current quantity on hand: 1,452.) To accommodate the EEPROM I used a zero-insertion force (ZIF) socket (DigiKey #A305-ND). Due to the tight spacing of chips on the board, I had to use a spare 32-lead IC socket as a spacer to provide the elevation needed to allow the ZIF socket to clear the 8255.

I set jumper P3 to "ROM" (standard setting), and P5 to "512K". Counterintuitively I had to set P4 to "32K" (not 512k) to get the EEPROM to be recognized. One other lesson learned from board construction was to face the larger opening of the piezoelectric speaker outward (the solder lug orientation suggested the opposite arrangement; however this blocks most of the sound.)

I used the SparkFun Serial Basic breakout board for the console serial line. The board is automatically powered from the SparkFun adapter so for most of my testing I did not use battery power. I used the larger raised tactile switches (DigiKey #450-1940-ND) along with 3D printed keycaps from Paul Schmidt.

Assembling the Code and Programming the EEPROM

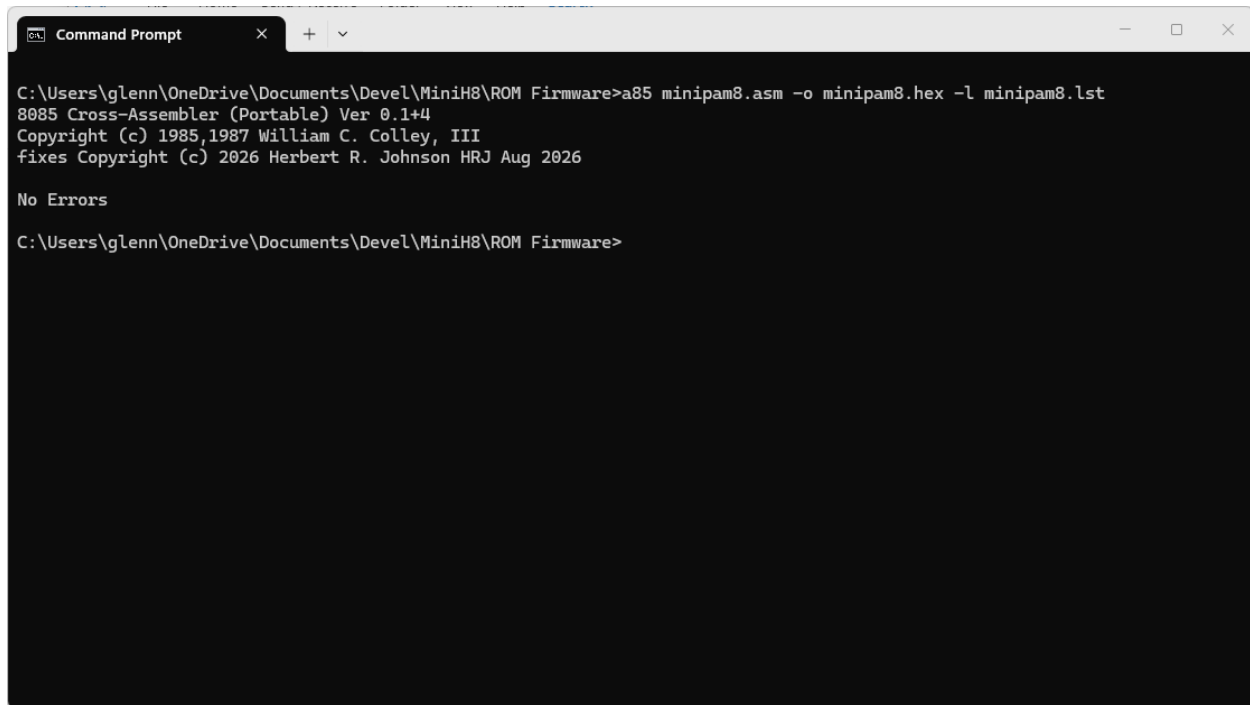
Software development was done using the A85 assembler from Herb Johnson. I ran this on my Windows 11 laptop. One useful feature that A85 seems to lack is the ability to check at time of assembly for errors in addressing. The Heath source code for PAM/8 uses the `ERRNZ` pseudo-op for this. For example, the line

```
ERRNZ    CTLFLG-MFLAG-1
```

is a way to ensure that the `CTLFLG` location is one more than the `MFLAG` location. This is a very useful feature for documenting code addressing dependencies. In this example the code uses the `HL` pair to point to `MFLAG` and then does an `INX H` to access `CTLFLG`. This can lead to subtle errors if, for example, the location of one or both of these were changed. In such a case the `ERRNZ` pseudo-op would catch that and report it.

I used the Notepad++ text editor to edit the code.

I ran the A85 assembler from a Command Prompt window:

A screenshot of a Windows Command Prompt window. The title bar says "Command Prompt". The command entered is "a85 minipam8.asm -o minipam8.hex -l minipam8.lst". The output shows the version "8085 Cross-Assembler (Portable) Ver 0.1+4", copyright information for William C. Colley, III and Herbert R. Johnson, and a confirmation "No Errors". The prompt then returns to the command line.

```
Command Prompt
C:\Users\glenn\OneDrive\Documents\Devel\MiniH8\ROM Firmware>a85 minipam8.asm -o minipam8.hex -l minipam8.lst
8085 Cross-Assembler (Portable) Ver 0.1+4
Copyright (c) 1985,1987 William C. Colley, III
fixes Copyright (c) 2026 Herbert R. Johnson HRJ Aug 2026

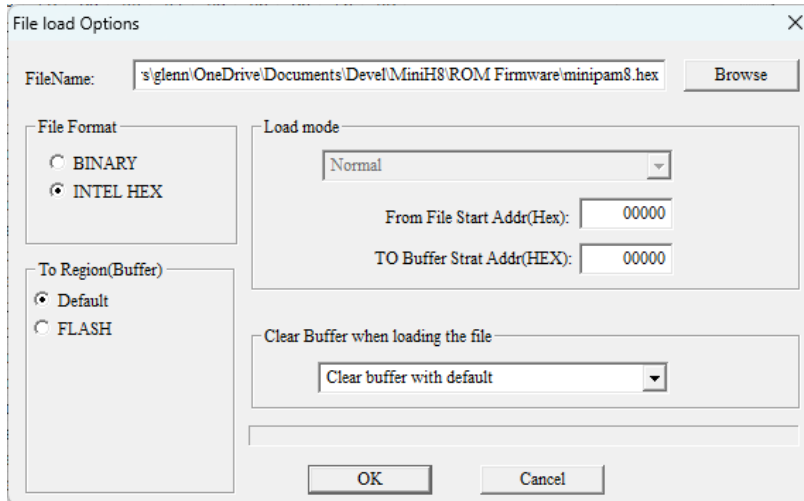
No Errors

C:\Users\glenn\OneDrive\Documents\Devel\MiniH8\ROM Firmware>
```

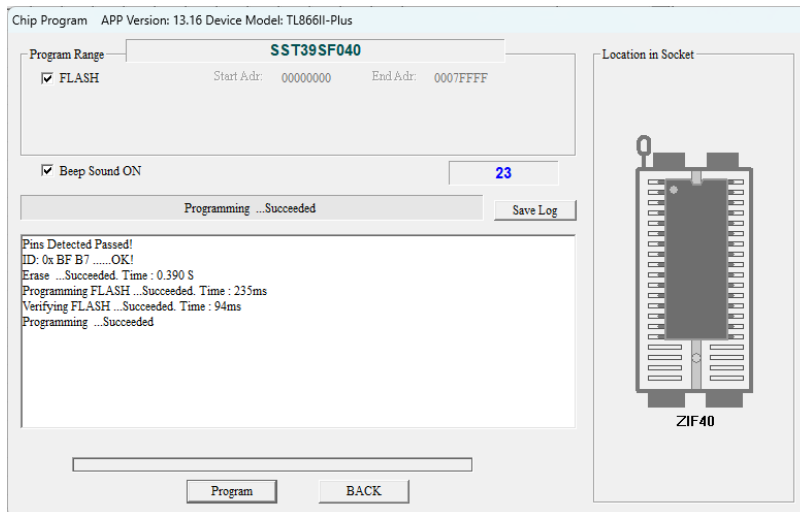
I used my TL866II+ programmer to program the EEPROM. Rebuilding the firmware and reprogramming the EEPROM took only seconds. The process was as follows:

1. Edit the code in Notepad++ and save it (Ctrl-S).
2. Use the up arrow in the Command Prompt window to recall the A85 command and reissue it to assemble the code (producing a `.LST` and `.HEX` file).

3. Insert the EEPROM in the TL866II+ programmer's ZIF socket.
4. Load the code from the Intel HEX file generated by the assembler:



5. Select "PROG" to program the EEPROM:



6. Remove the EEPROM and reinsert it in the ZIF socket on the H8 Mini, then power up the H8 Mini for testing.

As the Command Prompt tool keeps track of command history, all I had to do on subsequent assemblies was use the up-arrow key to reissue the A85 command.

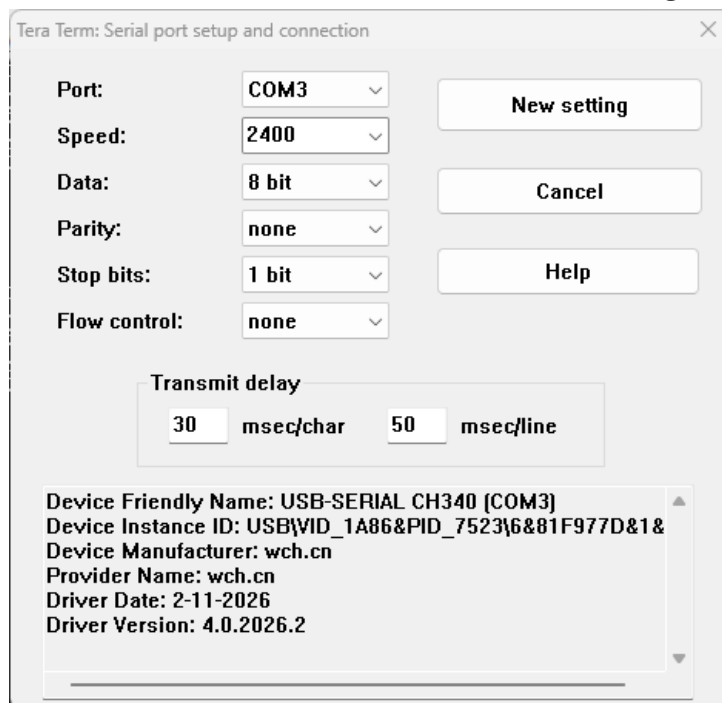
I would keep both the MINIPAM8.ASM and MINIPAM8.LST files open in Notepad++. If the assembly reported errors, I would select the MINIPAM8.LST tab (Notepad++ automatically notices that the window needs to be refreshed with the new version and prompts to do that), then scan for errors. Since error messages all begin with an uppercase letter in column 1 it is easy to search for them via the Regular Expression search feature in Notepad++. Searching for "`^[A-Z]`" finds all occurrences of an uppercase letter in column 1.

Creating MINIPAM8

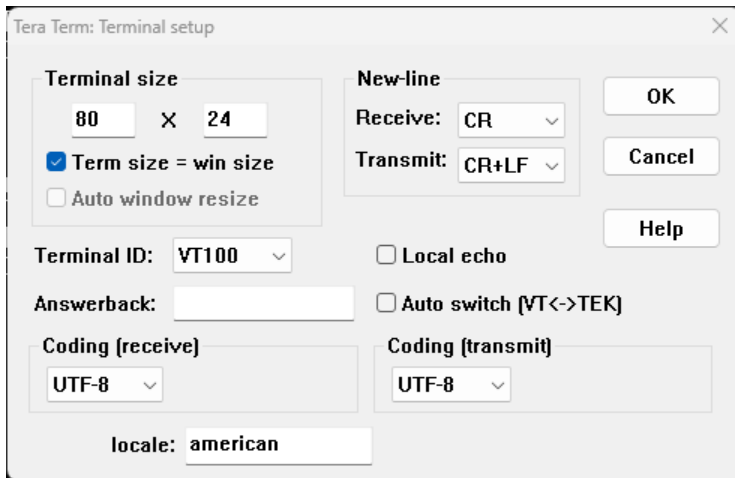
As of this writing, the creation of the new MINIPAM8 firmware is still a work in progress. I began with a version of the original Heath PAM/8 source code, which has been converted to A85 format by Herb Johnson. I verified that this code does indeed produce the exact image in the PAM/8 ROM.

The second important component was Jon Chapmans Glitch Works Monitor GWMON-80. I began with just GWMON so that I had a working console monitor. Since this monitor used bit-banged serial I found I had to tweak some of the serial timing parameters and slow things down to 2400 BAUD. These changes are documented in the code comments. With GWMON running I was able to download and run simple programs on the H8 Mini. For my console terminal I used TeraTerm version 4.105.

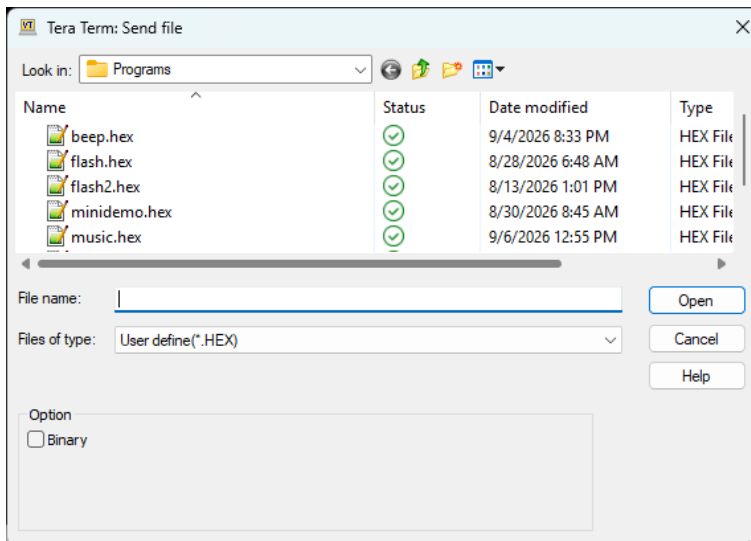
The GWMON monitor has a very useful “L” (LOAD) command, which allows you to send Intel HEX files to the H8 Mini. Here is how I configured the Serial port setup window:



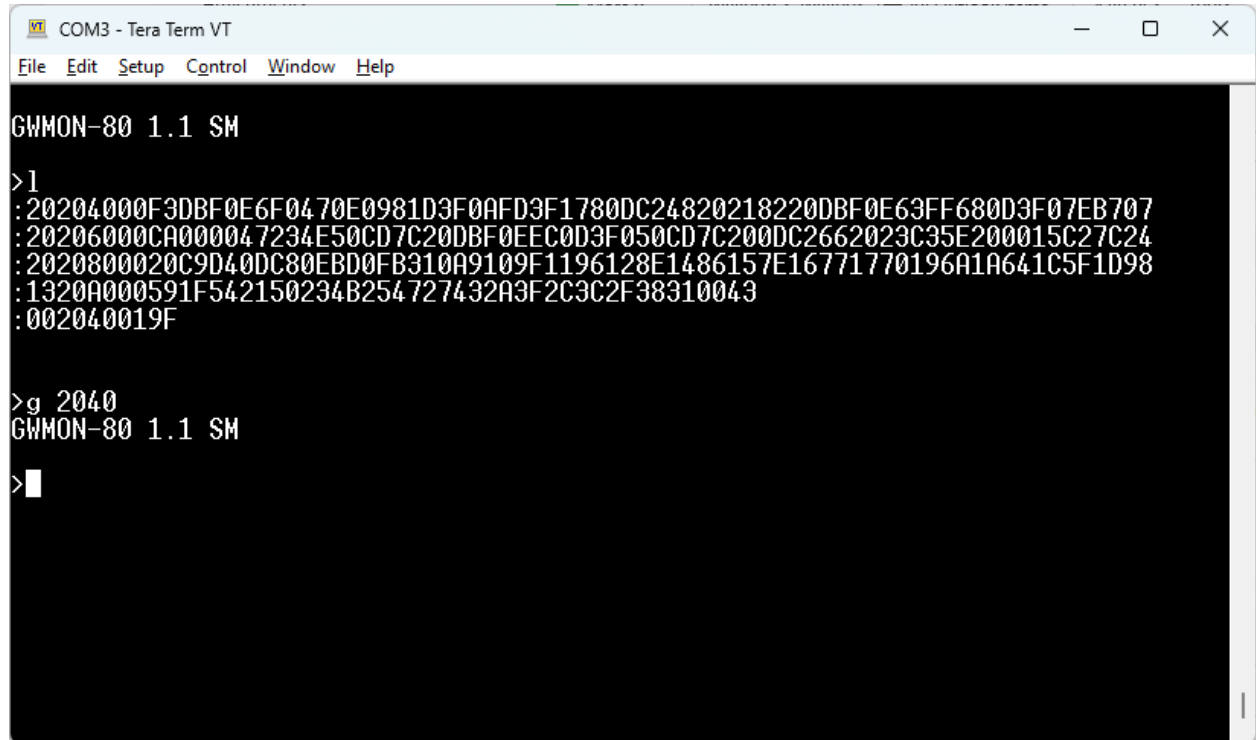
I also found that the LOAD command expects line feeds at the end of each line so I had to specify that in the terminal setup window:



I used the LOAD command to download and test software modules. This way I was able to incrementally test capabilities before folding them into the monitor. After issuing a "L" command I would invoke "File..." and "Send File..." from the TeraTerm window:



I then selected the file, hit Open and the download would proceed. The program could then be run via the monitor's "G" command:



```
COM3 - Tera Term VT
File Edit Setup Control Window Help

GWMON-80 1.1 SM

>1
:20204000F3DBF0E6F0470E0981D3F0AFD3F1780DC24820218220DBF0E63FF680D3F07EB707
:20206000CA000047234E50CD7C20DBF0EEC0D3F050CD7C200DC2662023C35E200015C27C24
:2020800020C9D40DC80EBD0FB310A9109F1196128E1486157E16771770196A1A641C5F1D98
:1320A000591F542150234B254727432A3F2C3C2F38310043
:002040019F

>g 2040
GWMON-80 1.1 SM

>█
```

Merging PAM/8 with GWMON

I have chosen to ORG PAM/8 at 0000H and GWMON at 0400H (1K). Currently the PAM/8 code provides interrupt-time processing for the front panel (e.g. LED refresh and keypad reading) and the GWMON provides the task time console functionality. Eventually PAM/8 will provide both, with an option to jump to the console via entry at 0400H.

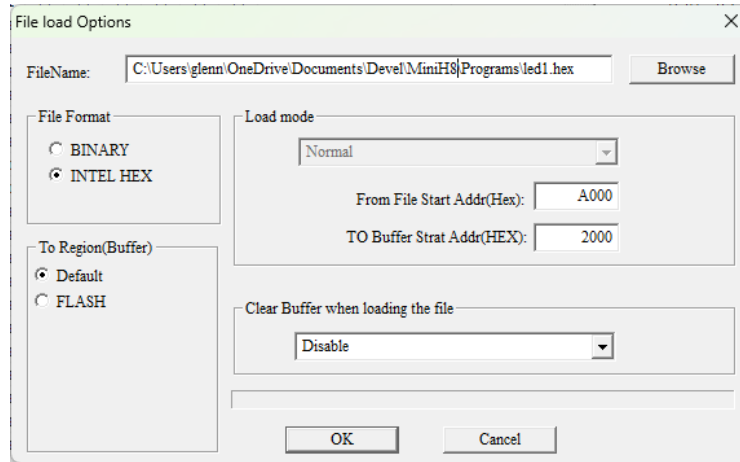
Executing Programs from ROM

The H8 Mini memory map mirrors the entire lower 32K of ROM into the upper 32K. This means, for example, that the memory from 8000H to 9FFFH is a duplicate of the memory from 0000H to 1FFFH.

This architecture allows for up to 24K of ROM-based software to reside in memory beginning at A000H. Placing a program in this space requires a few steps:

1. ORG the program at the address from which it will be executed (e.g. A000H).
2. When programming the EEPROM first load the monitor code as normal, then layer the ROM-based code on top as follows (this example is for use with the Xgpro software):
 - a. Set the "From File Start Address" to A000H

- b. Set the “To Buffer Start Address” to 2000H
- c. Be sure to select “Disable” under the “Clear buffer when loading” option



3. Though your code will appear at an offset of 2000H in the EEPROM, this will show up at A000H when run from the H8 Mini.
4. You can then execute your code via G A000 in the GWMON monitor.

Using the Front Panel LEDs

As of this writing (9/25/26) the MINIPAM8 monitor starts up in GWMON mode, that is, all interaction is through monitor commands and a serial terminal. The interrupt-time code to refresh the front panel LEDs is, however, active. By default, this starts up in memory display mode (and displaying location 040033A, which is the bottom byte of the TICCNT 2ms. timer), however what is being displayed can be changed by modifying the appropriate locations in low RAM. This is useful, for example, in debugging operations. You can use the GWMON’s ‘E’ command to change these values:

1. DSPMOD (2007H) controls the display mode. Set it to 00H for memory display or 02H for register display.
2. ABUSS (2014H) controls the memory location displayed when in memory mode. This is initiated to 040033A (201BH) when the H8 Mini is powered on, but you can change this to be any desired memory location.
3. REGI (2005H) controls what register is displayed when the display is in register mode, as follows:

Register to display	Value in 2005H
SP	00H

AF	02H
BC	04H
DE	06H
HL	08H
Pc	0AH

Interrupt-driven Serial I/O

The original GWMON code used simple polling for serial input. Once the PAM/8 interrupt-driven front panel service code was introduced, the polling approach no longer worked reliably. The front panel ISR steals enough CPU time that characters were getting dropped or garbled. Fortunately, the H8 Mini hardware was designed to support interrupt-driven serial operation. The RX serial line is inverted and fed directly to both the SID and RST 6.5 CPU inputs. This means that when a start bit arrives (RX goes low) the SID and RST 6.5 lines will go high, triggering an interrupt.

The serial interrupt service routine (ISR) is labelled CINT. It performs two tasks: 1) decipher the incoming byte by sampling at the appropriate intervals, and 2) store the byte in a buffer area. It is the job of the task-time code to check the buffer frequently enough to keep it from overflowing. The buffer itself consists of a one-byte counter (number of bytes available to be read from the buffer) followed by the actual buffer storage. The next character to be processed is always the first one in the buffer (i.e. the byte following the counter byte). Currently CINT discards any overflow if the buffer is full.

The corresponding task-time code to read these characters is CINNE. It does a blocking read (wait continuously until a character is available) and then pulls the incoming byte out of the buffer, decrements the buffer count, and moves all the remaining bytes up one position.

By default the monitor uses a 2-byte buffer in low RAM, but the location and size of the buffer are configurable by changing the values stored in INBADDR (address of buffer), and INBLEN (buffer size). Obviously, any changes to these values must be done with interrupts disabled:

```

INBADDR    DW    $INBUF      ; points to location of input buffer
INBLEN     DB    $INBUFL     ; max length of buffer
$INBUF     DB    0           ; input buffer count
           DS    2           ; very small buffer
$INBUFL    EQU    *-$INBUF-2 ; max length of buffer

```

This approach lets us relocate the \$INBUF buffer into user code, e.g. to allow for a larger buffer if needed, or for other purposes.

On the 8085 the RST 7.5 interrupt (which, in the H8 Mini, services the 2ms. Clock) has highest priority, however we want the RST 6.5 handler to override that, if necessary, so that characters don't get lost. This is handled by inserting code at the beginning of the RST 7.5 ISR to explicitly allow the lower priority interrupt to "interrupt the interrupt":

```
;
; We need to do nested interrupt handling here, allowing the lower
; priority (RST 6.5) serial line IRQ to interrupt the front panel
; refresh. Without this serial data will be lost. First enable interrupts
; and then specifically mask out the 7.5 ones to allow lower ones in
;
EI                ; ok to interrupt us
MVI    A,00001101B ; enable RST 6.5 - serial start bit IRQ
SIM                    ; apply the mask
```

Running Legacy Software

The H8 Mini is functionally similar to the very original H8 introduced in 1977 (which is no accident as it was created to celebrate the upcoming 50th anniversary of that event). Back then, to load and run software you could either key it in via the front panel (time consuming and error prone!) or, more likely, load it from cassette tape or an H10 paper tape reader. There was no operating system. There was only one console I/O standard - the H8-5 serial board (typically connected to a teletype or H9 terminal). In addition to the PAM/8 Panel Monitor (firmware in the H8), Heath distributed a debugger (BUG-8), editor (TED-8), assembler (HASL-8) and Benton Harbor BASIC.

Heath's chief software guru, J. Gordon Letwyn, knew that users would want to write their own programs to interact with the console, so he created "CONSL" - a standardized console front end. This code is about 300 bytes long and provides a standard API for interacting with the console and, to some extent, the cassette tape. This same CONSL code is the beginning code for all the Heath tools, plus the console-oriented games they distributed.

It would be very nice to be able to use those tools on the H8 Mini. Theoretically, this means replacing the CONSL piece with an equivalent one written for the H8 Mini serial hardware (which, of course, is completely different from the H8-5).

I decided to test this approach by converting one of the old H8 games: Hangman. This was distributed on audio cassette (Heath part number PA-84, see Catalog #842, p. 68:

https://heathkit.garlanger.com/catalogs/1978/Heathkit_Catalog_842.pdf) and has been

captured in H8T format as H8HANGM.H8T (See:
[https://sebhc.github.io/sebhc/software.html#Cassette Tape Images](https://sebhc.github.io/sebhc/software.html#Cassette_Tape_Images))

Source code was never published for this; however, I wanted to understand how the code interacts with the CONSL driver so I chose to disassemble and comment the H8HANGM code. Here is what I conclude from that investigation:

1. There are two vectors at the beginning of CONSL that must be patched to specify the correct jump locations for CONFIG and RESTART. The correct addresses must be identified by first dumping the original app and noting them. For example, here are the locations for the H8HANGM game (060037A and 041267A respectively):

```

                                00186  **      Program Entry Points
                                00187
040.100  303 037 060 00188  ENTRY  JMP    CONFIG
040.101                00189  CSTART EQU    *-2          cold start
                                00190
040.103  303 267 041 00191  RTN22  JMP    RESTART        warm start
```

These are easily found as they are right at the beginning of the code.

2. The CONSL application exposes methods for reading, writing and initializing the console:

```

040.144  303 255 040 00232  $RCHAR JMP    $RCHAR.      Read character
                                00233
040.147  303 332 040 00234  $WCHAR JMP    $WCHAR.      Write character
                                00235
040.152  303 367 040 00236  $PRSCL JMP    $PRSCL.      Preset Console
```

3. I found that the H8HANGM application mostly used these routines, however \$RCHAR is a blocking read function (it will wait forever for a character to be typed) so the H8HANGM application also sometimes looks directly at the interrupt-driven character buffer:

```

                                00238  **      Data and Buffers
                                00239  *
040.155  000                00240  $INBUF DB    0        Input buffer count
040.156                00241  DS     30        Type ahead buffer.
```

It does this when it needs to test if a character is available (checking if the byte at \$INBUF is zero) but it also does this to “dump” any characters that have been typed (by writing zero to that byte). This means that in our implementation of CONSL for the H8 Mini we will have to redirect the interrupt-driven buffer to this location by changing INBADDR and INBLEN.

4. The CONSL front end also standardizes the entry locations for port access. This made sense back then because all I/O was by reading and writing from ports. I did find code that called at least one of these so they can be used by applications. Most of these are 3-byte entries, so we can put in a JMP instruction (e.g. because our input or output via the 8255 requires more than a simple IN or OUT), however for \$CDIN (Console Data In) that will not work as you must guarantee that the third byte is a RET instruction. There are locations in H8HANGM that jump to this spot when they require a RETURN.

```

00193  **      Port Routines.
00194  *
00195  *      All programs must use these
00196  *      routines, to allow port address
00197  *      changeability.
040.106 333 372 00198 $CDIN IN IP.CDP Console Data In
040.110 311 00199 $RET RET

```

For example: H8HANGM uses the \$RET location in its Control-C handler:

```

01083  **      CINIT - Console initialization
01084  *
01085  *      Set CC handler to just RETURN and
01086  *      clear console bits
01087  *
051.243 041 110 040 01088 CINIT LXI H,$RET
051.246 042 250 040 01089 SHLD $CSIC
051.251 257 01090 XRA A
051.252 062 252 040 01091 STA $CSLCTL
051.255 311 01092 RET

```

I think for the most part we can ignore these port calls (maybe set the port numbers to 000Q ?) and hope that none of the applications use them directly. This will take some experimentation.

In order to run these legacy applications we will need to replace the CONSL front end with our own custom version. Here are the steps needed:

1. Using the Heath sample CONSL code as a guide, develop a functionally equivalent version for the H8 Mini and assemble it to create the binary machine code. As all of the legacy applications have the CONSL code as the front end we will need to replace the legacy CONSL code with our new version.
2. The legacy applications are stored in H8T format (a direct digital expression of the original cassette data stream), and the H8T format uses a checksum to detect download errors, so we can't edit them directly, so we will first convert them from

into an editable binary form. I have written a program HTOA.C which converts H8T formatted files into Heath's .ABS format.

3. Once the legacy application is in .ABS format we will use a binary editing program (probably on the PC side which is easier, e.g. with a tool such as the HxD Hex Editor) to overlay the new CONSL code on top of the old. It will be necessary to take care not to change the locations of any of the code that follows CONSL, i.e. the substation must be for the exact number of bytes in CONSL.

One thing to be aware of is that Heath released updates to their applications that changed the CONSL code. This was done to allow support for multiple serial I/O boards. When the H8 was originally introduced the only serial option was the H-8-5, which uses the Intel 8251 Programmable Communication Interface chip, however later Heath released the H-8-4 board, which used the Intel 8250 Universal Asynchronous Receiver/Transmitter (UART) chip. At some point Heath updated CONSL to automatically detect which serial interface board was in use. I have not yet fully explored this. For now my work is focused on the original 8251-only releases.

4. Once the patched .ABS version of the program is done we will need to convert this back to a form that can be loaded into the H8 Mini. Currently that means Intel Hex format. Fortunately there is a HUG disk (885-1089) that contains utilities to convert between .ABS and .HEX format.
5. Once the patched application is in .HEX form we can load it using the GWMON 'L' command and then 'G 2040' to execute.